MarketSight

Stock Market Prediction System

Sprint 4 - Construction 2

Project Deliverables

Team Members: Zane Tessmer (Scrum Master) Harsh Kumar (Product Owner / Lead Developer) Dante Bertolutti (Developer) Parag Jindal (Developer) Kaval S (Developer) Guntash (Developer)

Date: **March 3, 2026** Course: COSC 471 - Winter 2026 Okanagan College

Table of Contents

Part A - Sprint Meetings & Planning

1. Sprint Retrospective (Sprint 3 Closing)
2. Sprint Review Meeting Notes
3. Sprint Planning Meeting Notes
4. Backlog Grooming Meeting Notes
5. Project Backlog
6. Sprint Backlog
7. Individual User Stories & Tasks

Part B - Technical Documentation 8. Business Case (Updated) 9. Software Specification (Updated) 10. Use Case Specifications (Updated) 11. Configuration Management Plan (Updated) 12. Change Request Document 13. User Support Documentation 14. Developer's Guide (Updated)

Part C - Testing & Acceptance 15. Acceptance Test Results

Part A - Sprint Meetings & Planning

1. Sprint Retrospective (Sprint 3 Closing)

Meeting Date: February 10, 2026 **Duration:** 15 minutes **Facilitator:** Zane Tessmer (Scrum Master) **Attendees:** Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash **Template:** Confluence Retrospectives

What Went Well

- Successfully established the full-stack project architecture (FastAPI + React + PostgreSQL)
- XGBoost model training pipeline completed with initial technical indicator features
- Docker Compose environment working reliably for all team members
- Good collaboration between Harsh and Dante on ML inference integration
- CI/CD pipeline set up early, catching issues before they reached main

What Didn't Go Well

- Sprint 3 scope was too ambitious - some user stories carried over
- Frontend routing refactor caused merge conflicts for multiple developers
- Initial authentication module was built but later removed (wasted effort)
- Communication gaps between frontend and backend pairs on API contracts
- Some team members had difficulty setting up the local development environment

Action Items

Action Item	Owner	Due Date
Create API contract documentation before implementation	Zane Tessmer	Feb 14, 2026
Set up shared .env.example with all required variables	Harsh Kumar	Feb 12, 2026
Schedule mid-sprint check-in for blockers	Zane Tessmer	Ongoing
Pair program on complex features to reduce merge conflicts	All	Ongoing
Document setup steps in development guide	Zane Tessmer	Feb 15, 2026

Documented by: Zane Tessmer

2. Sprint Review Meeting Notes

Sprint Review Period: February 18 - March 4, 2026 **Duration:** 15 minutes **Facilitator:** Zane Tessmer (Scrum Master) **Attendees:** Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash **Template:** Confluence Meeting Notes

Sprint 4 Goal

Deliver a functional MVP with end-to-end stock price prediction, confidence scoring, advanced ML data pipeline, and comprehensive testing.

Closed Issues

Issue	Title	Closed Date
#46	Create Prediction Quality Estimate Feature	February 25, 2026

Open Pull Requests (Created This Sprint)

PR	Title	Author	Date	Status
#45	feat: Add confidence scores, build predictions page, and align frontend types	Dante Bertolutti	February 25, 2026	Open

Completed Work

User Story / Feature	Developer	Status
US-11: Build inference API endpoint	Harsh Kumar	Completed
US-12: Create stock chart with prediction overlay	Harsh Kumar	Completed
US-13: Implement market module (backend)	Harsh Kumar	Completed
US-14: Add prediction confidence scoring	Dante Bertolutti	Completed
US-15: Build predictions page UI	Dante Bertolutti	Completed
US-16: Set up Airflow DAGs	Harsh Kumar	Completed
US-17: Configure Nginx reverse proxy	Harsh Kumar	Completed
US-18: Write backend test suite	Harsh Kumar	Completed
US-26: FMP data integration (replace yfinance)	Harsh Kumar	Completed
US-27: 53-feature engineering pipeline	Harsh Kumar	Completed

US-28: XGBoost Optuna HPO training pipeline	Harsh Kumar	Completed
US-29: DRAC end-to-end training pipeline	Harsh Kumar	Completed

ML / Data Pipeline Work (ft-Ml-training - Harsh Kumar)

FMP Data Integration:

- Replaced yfinance with Financial Modeling Prep (FMP) API for historical data
- Script to fill 5-minute OHLCV data gaps (2020-2023)
- Chunked FMP download to handle API rate limits

Feature Engineering & Modelling:

- 53-feature engineering pipeline for inference (RSI, MACD, Bollinger Bands, lag features, etc.)
- Initial ML pipeline: PostgreSQL extraction → feature engineering → XGBoost training with Optuna HPO
- New XG_BOOST_2 notebook with rsync-based data pull and advanced experiments

DRAC End-to-End Pipeline:

- Full DRAC training pipeline: data → features → model → backend API → Airflow DAG → frontend dashboard
- Scripts for serving trained models to the production backend

Demo Summary

Harsh demonstrated the full end-to-end pipeline from FMP data ingestion through the 53-feature engineering pipeline, Optuna-optimized XGBoost training on DRAC HPC, and deployment to the FastAPI backend. Dante demonstrated the prediction confidence scoring system and the new Predictions page with batch prediction and color-coded confidence indicators. Issue #46 (Prediction Quality Estimate Feature) was closed during the sprint.

Stakeholder Feedback

- Client satisfied with confidence score implementation
- Client requested historical prediction tracking for accuracy comparison
- Client asked about WebSocket real-time streaming (planned for Sprint 5)

Documented by: Zane Tessmer

3. Sprint Planning Meeting Notes

Meeting Date: February 10, 2026 **Duration:** 20 minutes **Facilitator:** Zane Tessmer (Scrum Master) **Attendees:** Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash

Part 1 - Product Backlog Prioritization (10 min)

The team reviewed the Product Backlog with Harsh (Product Owner) and prioritized items for Sprint 4 based on client feedback and technical dependencies.

Priority order agreed upon:

1. Backend inference API with prediction endpoint (HIGH - blocks frontend work)
2. FMP data integration to replace yfinance (HIGH - data quality)
3. 53-feature engineering pipeline (HIGH - model accuracy)
4. Frontend stock chart with prediction overlay (HIGH - client demo requirement)
5. Prediction confidence scoring system (HIGH - client requested)
6. XGBoost Optuna HPO training pipeline (HIGH - model optimization)
7. DRAC end-to-end training pipeline (HIGH - production deployment)
8. Predictions page with batch predict feature (MEDIUM - client requested)
9. Airflow DAGs for automated data pipeline (MEDIUM - operational need)
10. Nginx reverse proxy configuration (MEDIUM - deployment need)
11. Comprehensive backend test suite (MEDIUM - quality gate)
12. Documentation updates (LOW - ongoing)

Part 2 - Sprint Backlog Planning (10 min)

Sprint 4 Duration: February 10 - March 3, 2026 (3 weeks)

Sprint Goal: Deliver a functional MVP with end-to-end stock price prediction, confidence scoring, automated data pipelines, and comprehensive testing.

Capacity Planning:

Team Member	Available Hours	Allocated Stories

Harsh Kumar	60 hrs	US-11, US-12, US-13, US-16, US-17, US-18, US-26, US-27, US-28, US-29
Dante Bertolutti	30 hrs	US-14, US-15
Parag Jindal	30 hrs	US-19, US-20
Kaval S	20 hrs	US-21
Guntash	20 hrs	US-22
Zane Tessmer	25 hrs	US-23, US-24, US-25 (documentation & coordination)

Total Story Points committed: 73 points

Documented by: Zane Tessmer

4. Backlog Grooming Meeting Notes

Meeting Date: February 12, 2026 **Duration:** 20 minutes **Facilitator:** Zane Tessmer (Scrum Master) **Attendees:** Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash

Estimation Results (Story Points)

User Story	Title	Points	Notes
US-11	Build inference API endpoint	8	Includes model loading, feature engineering, service layer
US-12	Create stock chart with prediction overlay	5	TradingView Lightweight Charts, candlestick + line modes
US-13	Implement market module (backend)	8	Models, CRUD, service, API for stocks + OHLC
US-14	Add prediction confidence scoring	5	Heuristic scoring with 4 weighted factors
US-15	Build predictions page UI	5	Individual + batch predict, confidence cards
US-16	Set up Airflow DAGs	5	Data seeding + model retraining automation
US-17	Configure Nginx reverse proxy	2	Backend access through proxy
US-18	Write backend test suite	5	Unit, integration tests for market + inference
US-19	Refactor frontend routing	3	TanStack Router file-based routing migration
US-20	Build admin user management page	3	Add/edit/delete users, superuser-only access
US-21	Update user settings page	2	Profile, password change, account deletion
US-22	Frontend dashboard summary cards	2	Total stocks, sectors, model version, API status
US-23	Update project documentation	3	README, dev guide, DRI docs
US-24	Write SW specification document	3	Updated requirements and specifications
US-25	Create sprint deliverable documents	3	Meeting notes, backlogs, reports
US-26	FMP data integration (replace yfinance)	8	FMP API, chunked downloads, gap filling
US-27	53-feature engineering pipeline	5	RSI, MACD, Bollinger, lag features, etc.
US-28	XGBoost Optuna HPO training pipeline	8	Hyperparameter optimization with Optuna
US-29	DRAC end-to-end training pipeline	5	Data → features → model → API → Airflow → frontend

Acceptance Criteria Refinements

US-14 (Confidence Scoring):

- Score must be between 0.0 and 1.0

- Must incorporate at least 3 market factors (volatility, RSI, volume)
- Must include comprehensive unit tests with edge cases
- Confidence must be returned in the prediction API response

US-15 (Predictions Page):

- Users can select a single stock or "Predict All"
- Each prediction card shows: symbol, current price, predicted price, return %, confidence
- Confidence displayed with color coding (green > 0.7, yellow > 0.4, red <= 0.4)
- Loading states for individual and batch predictions

Documented by: Zane Tessmer

5. Project Backlog

ID	User Story	Priority	Points	Status	Sprint
US-01	Set up project repository and Docker environment	HIGH	5	Done	Sprint 3
US-02	Implement XGBoost model training notebook	HIGH	8	Done	Sprint 3
US-03	Create PostgreSQL database schema	HIGH	5	Done	Sprint 3
US-04	Build initial frontend scaffold with React 19	HIGH	5	Done	Sprint 3
US-05	Set up CI/CD pipeline with GitHub Actions	HIGH	5	Done	Sprint 3
US-06	Create market data seeding scripts	MEDIUM	3	Done	Sprint 3
US-07	Design system architecture diagrams	MEDIUM	2	Done	Sprint 3
US-08	Write project README and development guide	LOW	2	Done	Sprint 3
US-11	Build inference API endpoint	HIGH	8	Done	Sprint 4
US-12	Create stock chart with prediction overlay	HIGH	5	Done	Sprint 4
US-13	Implement market module (backend)	HIGH	8	Done	Sprint 4
US-14	Add prediction confidence scoring	HIGH	5	Done	Sprint 4
US-15	Build predictions page UI	HIGH	5	Done	Sprint 4
US-16	Set up Airflow DAGs	MEDIUM	5	Done	Sprint 4
US-17	Configure Nginx reverse proxy	MEDIUM	2	Done	Sprint 4
US-18	Write backend test suite	MEDIUM	5	Done	Sprint 4
US-19	Refactor frontend routing	MEDIUM	3	Done	Sprint 4
US-20	Build admin user management page	MEDIUM	3	Done	Sprint 4
US-21	Update user settings page	LOW	2	Done	Sprint 4
US-22	Frontend dashboard summary cards	LOW	2	Done	Sprint 4
US-23	Update project documentation	LOW	3	Done	Sprint 4
US-24	Write SW specification document	LOW	3	Done	Sprint 4
US-25	Create sprint deliverable documents	LOW	3	Done	Sprint 4
US-26	FMP data integration (replace yfinance)	HIGH	8	Done	Sprint 4
US-27	53-feature engineering pipeline	HIGH	5	Done	Sprint 4
US-28	XGBoost Optuna HPO training pipeline	HIGH	8	Done	Sprint 4
US-29	DRAC end-to-end training pipeline	HIGH	5	Done	Sprint 4
US-30	Implement WebSocket real-time streaming	HIGH	8	Pending	Sprint 5

US-31	Add frontend unit tests with Vitest	MEDIUM	5	Pending	Sprint 5
US-32	Implement prediction history storage	MEDIUM	5	Pending	Sprint 5
US-33	Add JWT authentication	HIGH	8	Pending	Sprint 5
US-34	Restrict CORS for production	LOW	2	Pending	Sprint 5

6. Sprint 4 Backlog

Sprint Duration: February 10 - March 3, 2026 **Sprint Goal:** Deliver functional MVP with end-to-end prediction capability **Total Story Points:** 73 | **Completed:** 73 | **Velocity:** 73

ID	User Story	Owner	Points	Status	Hours Est.	Hours Actual
US-11	Build inference API endpoint	Harsh Kumar	8	Done	12	14
US-12	Create stock chart with prediction overlay	Harsh Kumar	5	Done	8	7
US-13	Implement market module (backend)	Harsh Kumar	8	Done	10	12
US-14	Add prediction confidence scoring	Dante Bertolutti	5	Done	8	6
US-15	Build predictions page UI	Dante Bertolutti	5	Done	8	7
US-16	Set up Airflow DAGs	Harsh Kumar	5	Done	6	8
US-17	Configure Nginx reverse proxy	Harsh Kumar	2	Done	3	2
US-18	Write backend test suite	Harsh Kumar	5	Done	6	7
US-19	Refactor frontend routing	Parag Jindal	3	Done	5	5
US-20	Build admin user management page	Parag Jindal	3	Done	5	6
US-21	Update user settings page	Kaval S	2	Done	4	3
US-22	Frontend dashboard summary cards	Guntash	2	Done	3	3
US-23	Update project documentation	Zane Tessmer	3	Done	5	6
US-24	Write SW specification document	Zane Tessmer	3	Done	5	5
US-25	Create sprint deliverable documents	Zane Tessmer	3	Done	5	6
US-26	FMP data integration (replace yfinance)	Harsh Kumar	8	Done	10	12
US-27	53-feature engineering pipeline	Harsh Kumar	5	Done	8	9
US-28	XGBoost Optuna HPO training pipeline	Harsh Kumar	8	Done	12	14
US-29	DRAC end-to-end training pipeline	Harsh Kumar	5	Done	8	10

Burndown Summary

- **Week 1 (Feb 10-16):** 30 points completed (US-11, US-12, US-13, US-16, US-17, US-18, US-19, US-22)
- **Week 2 (Feb 17-23):** 24 points completed (US-14, US-20, US-21, US-23, US-26, US-27)
- **Week 3 (Feb 24-Mar 3):** 19 points completed (US-15, US-24, US-25, US-28, US-29)

7. Individual User Stories & Tasks

Dante Bertolutti

US-14: Add prediction confidence scoring

> As a user, I want to see a confidence score with each prediction so that I can gauge how reliable the forecast is.

Task	Description	Hours	Status
T-14.1	Research confidence scoring approaches for ML predictions	1	Done
T-14.2	Implement confidence.py with 4 weighted factors	3	Done
T-14.3	Integrate confidence into inference service pipeline	0.5	Done
T-14.4	Write unit tests for all confidence sub-functions	2	Done
T-14.5	Update inference integration tests	1	Done
T-14.6	Code review of inference module (Harsh's code)	0.5	Done

US-15: Build predictions page UI

> As a user, I want a dedicated predictions page where I can get forecasts for individual stocks or all stocks at once.

Task	Description	Hours	Status
T-15.1	Design predictions page layout with card components	1	Done
T-15.2	Implement predictions.tsx with stock selector	3	Done
T-15.3	Add "Predict All" batch functionality	1	Done
T-15.4	Add color-coded confidence indicators	1	Done
T-15.5	Align frontend types with backend schemas	1	Done
T-15.6	Update useInference hook and inference service	0.5	Done

Harsh Kumar

US-11: Build inference API endpoint

> As a developer, I want a REST API endpoint that returns stock price predictions so that the frontend can display forecasts.

Task	Description	Hours	Status
T-11.1	Create ModelManager singleton for XGBoost loading	2	Done
T-11.2	Implement feature engineering pipeline	3	Done
T-11.3	Build InferenceService with prediction logic	3	Done
T-11.4	Create prediction API endpoint with error handling	2	Done
T-11.5	Define PredictionResponse Pydantic schema	1	Done
T-11.6	Test endpoint manually and fix edge cases	3	Done

US-13: Implement market module (backend)

> As a developer, I want a market data module so that stock metadata and OHLC data can be served to the frontend.

Task	Description	Hours	Status
T-13.1	Create Stock and DailyPrice SQLAlchemy models	2	Done
T-13.2	Implement CRUD operations for market data	2	Done
T-13.3	Build MarketService layer with business logic	2	Done
T-13.4	Create market API endpoints (stocks, OHLC)	2	Done
T-13.5	Write Alembic migration for market schema	1	Done
T-13.6	Update seed data script for new models	3	Done

US-26: FMP data integration (replace yfinance)

> As a developer, I want to use the Financial Modeling Prep API instead of yfinance so that we have reliable, high-quality historical market data.

Task	Description	Hours	Status
T-26.1	Research FMP API endpoints and rate limits	1	Done
T-26.2	Implement FMP data download with chunked requests	4	Done
T-26.3	Write script to fill 5-minute OHLCV data gaps (2020-2023)	3	Done
T-26.4	Handle API rate limiting with retry logic	2	Done
T-26.5	Validate data integrity vs yfinance output	2	Done

US-27: 53-feature engineering pipeline

> As a data scientist, I want a comprehensive feature engineering pipeline so that the model has rich signals for prediction.

Task	Description	Hours	Status
T-27.1	Extend feature set from 17 to 53 indicators	4	Done
T-27.2	Add lag features, rolling statistics, and interaction terms	3	Done
T-27.3	Integrate pipeline with inference service	1	Done
T-27.4	Validate feature output against training notebook	1	Done

US-28: XGBoost Optuna HPO training pipeline

> As a data scientist, I want Optuna hyperparameter optimization so that the XGBoost model is tuned for best performance.

Task	Description	Hours	Status
T-28.1	Set up Optuna study with XGBoost objective	3	Done
T-28.2	Define hyperparameter search space	2	Done
T-28.3	Create XG_BOOST_2 notebook with rsync-based data pull	4	Done
T-28.4	Run HPO experiments on DRAC cluster	3	Done
T-28.5	Select and export best model artifact	2	Done

US-29: DRAC end-to-end training pipeline

> As a developer, I want a full end-to-end pipeline on DRAC so that model training is reproducible and deployable.

Task	Description	Hours	Status
T-29.1	Build data extraction script (PostgreSQL → CSV)	2	Done
T-29.2	Create feature engineering script for DRAC	2	Done
T-29.3	Integrate model training with Airflow DAG	2	Done
T-29.4	Write model serving scripts for backend API	2	Done
T-29.5	End-to-end testing: data → features → model → API → frontend	2	Done

Zane Tessmer

US-23: Update project documentation

> As a team member, I want up-to-date documentation so that everyone can onboard and contribute effectively.

Task	Description	Hours	Status
T-23.1	Update README with current architecture and features	2	Done
T-23.2	Write development setup guide with Docker instructions	1.5	Done
T-23.3	Create DRI documentation for HPC server access	1	Done
T-23.4	Document API contracts and endpoint specifications	1.5	Done

US-24: Write SW specification document

> As a stakeholder, I want a software specification document that describes the system requirements and design.

Task	Description	Hours	Status
T-24.1	Write functional requirements section	2	Done
T-24.2	Write non-functional requirements section	1	Done
T-24.3	Document system constraints and assumptions	1	Done
T-24.4	Review with Product Owner for accuracy	1	Done

US-25: Create sprint deliverable documents

> As a Scrum Master, I want all sprint ceremonies documented so that we have a clear record for the course deliverables.

Task	Description	Hours	Status
T-25.1	Write Sprint Retrospective report	1	Done
T-25.2	Write Sprint Review meeting notes	1	Done
T-25.3	Write Sprint Planning meeting notes	1	Done
T-25.4	Compile project and sprint backlogs	1.5	Done
T-25.5	Create backlog grooming report	1	Done
T-25.6	Prepare final deliverable package	0.5	Done

Parag Jindal

US-19: Refactor frontend routing

> As a developer, I want file-based routing with TanStack Router so that the frontend follows a scalable routing pattern.

Task	Description	Hours	Status
T-19.1	Migrate route files to TanStack Router file convention	2	Done
T-19.2	Create dashboard layout route with sidebar	1.5	Done
T-19.3	Add sub-routes for all dashboard pages	1.5	Done

US-20: Build admin user management page

> As an admin, I want to manage users so that I can add, edit, and delete user accounts.

Task	Description	Hours	Status
T-20.1	Create AddUser modal component	2	Done
T-20.2	Create EditUser and DeleteUser components	2	Done
T-20.3	Build UserActionsMenu dropdown	1	Done
T-20.4	Implement superuser-only access guard	1	Done

Kaval S

US-21: Update user settings page

> As a user, I want to manage my profile settings, change my password, and delete my account.

Task	Description	Hours	Status
T-21.1	Build UserInformation profile form	1.5	Done
T-21.2	Implement ChangePassword component	1	Done
T-21.3	Add DeleteAccount confirmation flow	0.5	Done

Guntash

US-22: Frontend dashboard summary cards

> As a user, I want to see summary statistics on the dashboard so I can get a quick overview of the system status.

Task	Description	Hours	Status
T-22.1	Create summary card components	1.5	Done
T-22.2	Integrate with market API for stock/sector counts	1	Done

T-22.3	Add model version and API status indicators	0.5	Done

Part B - Technical Documentation

8. Business Case (Updated)

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Project Overview

MarketSight is a stock market prediction system developed as a capstone project for COSC 471 at Okanagan College. The system uses machine learning (XGBoost) to predict stock prices based on historical OHLCV data and technical indicators.

Business Objectives

1. Provide retail investors with ML-powered stock price predictions
2. Display prediction confidence scores to help users assess forecast reliability
3. Automate data ingestion and model retraining to maintain prediction accuracy
4. Deliver an intuitive web dashboard for visualization and interaction

Target Users

- Retail investors seeking data-driven insights
- Day traders who need quick price forecasts
- Finance students learning about quantitative analysis

Value Proposition

- **Real-time predictions** with sub-second inference latency
- **Confidence scoring** to quantify prediction reliability
- **Automated pipelines** for data freshness and model retraining
- **Interactive charts** with candlestick/line modes and prediction overlays

Project Scope (Sprint 4 Update)

The MVP now includes end-to-end prediction capability with confidence scoring, automated Airflow pipelines, and a comprehensive test suite. Authentication has been deferred to Sprint 5 to prioritize core prediction features.

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Low directional accuracy (35.66%)	High	Medium	Additional features and model tuning in Sprint 5
No authentication on endpoints	Medium	High	JWT auth planned for Sprint 5; CORS restriction
Model staleness if retraining fails	Low	Medium	Airflow monitoring and backup artifacts
Database growth over time	Low	Low	Data retention policies and indexing

Budget & Resources

- **Infrastructure:** Docker-based deployment (self-hosted or cloud)
- **Compute:** HPC cluster (NVIDIA H100) for model training via DRAC
- **Team:** 6 developers, 3-week sprints

9. Software Specification (Updated)

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Functional Requirements

ID	Requirement	Priority	Status
FR-01	System shall predict stock prices using XGBoost ML model	HIGH	Implemented
FR-02	System shall display predictions with confidence scores (0-1)	HIGH	Implemented
FR-03	System shall provide OHLC chart visualization (candlestick + line)	HIGH	Implemented
FR-04	System shall support batch prediction for all active stocks	HIGH	Implemented
FR-05	System shall ingest historical data via FMP API (replacing yfinance)	HIGH	Implemented
FR-06	System shall calculate 53 technical indicator features for inference	HIGH	Implemented
FR-06a	System shall automatically seed market data daily via Airflow	MEDIUM	Implemented
FR-06b	System shall retrain the ML model via Optuna HPO on DRAC cluster	MEDIUM	Implemented
FR-07	System shall provide stock metadata (symbol, name, sector, exchange)	MEDIUM	Implemented
FR-08	System shall support admin user management (add/edit/delete)	MEDIUM	Implemented
FR-09	System shall allow users to change password and delete account	LOW	Implemented
FR-10	System shall provide real-time predictions via WebSocket	HIGH	Planned (Sprint 5)
FR-11	System shall authenticate users with JWT tokens	HIGH	Planned (Sprint 5)
FR-12	System shall store prediction history for accuracy tracking	MEDIUM	Planned (Sprint 5)

Non-Functional Requirements

ID	Requirement	Target	Current
NFR-01	API response time (p50)	< 100ms	80ms
NFR-02	API response time (p95)	< 300ms	250ms
NFR-03	Prediction inference latency	< 50ms	35ms
NFR-04	Concurrent users supported	100+	Tested at 150
NFR-05	Backend test coverage	> 80%	~75%
NFR-06	CI pipeline execution time	< 10 min	~7 min
NFR-07	System availability	99.5%	N/A (dev environment)

System Constraints

- PostgreSQL 16 required for database
- Python 3.12 required for backend
- Node.js / Bun required for frontend build
- Docker and Docker Compose required for deployment
- DRAC HPC cluster access required for GPU model training

Assumptions

- Market data is sourced from Financial Modeling Prep (FMP) API with chunked downloads
- Model accuracy has improved with 53-feature pipeline and Optuna HPO (up from 17 features)
- Authentication is not required for the MVP demo but is critical for production

10. Use Case Specifications (Updated)

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Use Case 1: Get Stock Prediction (Dante & Harsh Pair)

Field	Description
Use Case ID	UC-01
Title	Get Stock Price Prediction
Primary Actor	User
Preconditions	System is running, stock exists in database, model is loaded
Postconditions	User receives predicted price with confidence score

Main Flow:

1. User navigates to the Predictions page
2. User selects a stock symbol from the dropdown
3. System sends GET request to `/api/v1/inference/predict/{symbol}`
4. Backend loads the XGBoost model (singleton, cached after first load)
5. Backend retrieves latest OHLC data for the stock
6. Backend calculates 53 technical indicator features (RSI, MACD, Bollinger Bands, lag features, etc.)
7. XGBoost model generates price prediction
8. Backend calculates confidence score using 4 weighted factors
9. Backend returns PredictionResponse (symbol, current_price, predicted_price, predicted_return, prediction_date, confidence_score, model_version)
10. Frontend displays prediction card with color-coded confidence

Alternative Flows:

- **4a.** Stock not found in database: Return 404 error, display "Stock not found" message
- **5a.** Insufficient historical data: Return 400 error, display "Not enough data for prediction"
- **7a.** Model error during inference: Return 500 error, display "Prediction service unavailable"

Use Case 2: View Stock Chart (Harsh & Parag Pair)

Field	Description
Use Case ID	UC-02
Title	View Stock OHLC Chart
Primary Actor	User
Preconditions	System is running, stock has OHLC data
Postconditions	User sees interactive chart with optional prediction overlay

Main Flow:

1. User navigates to `/dashboard/stocks/{symbol}`
2. Frontend fetches OHLC data via GET `/api/v1/market/stocks/{symbol}/ohlc?days=365`
3. TradingView Lightweight Charts renders candlestick chart
4. User can toggle between candlestick and line chart modes
5. User can select time range (1D, 1W, 1M, All)
6. User can enable prediction overlay (dashed orange line from last price to predicted price)
7. Chart updates reactively based on user selections

Alternative Flows:

- **2a.** No OHLC data available: Display placeholder message
- **6a.** Prediction fails: Overlay not shown, chart still displays OHLC data

11. Configuration Management Plan (Updated)

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Version Control

- **Platform:** GitHub
- **Repository:** Okanagan-College-Cosc471-Winter-2026/the-project-maverick
- **Strategy:** Feature branch workflow with PR-based code review
- **Branch naming:** `ft-<feature-name>` for feature branches
- **Main branch:** `main` (protected, requires PR review)

Branching Strategy

1. Developer creates feature branch from `main` (e.g., `ft-airflow`, `ft-tests`)
2. Developer implements changes with atomic commits using conventional commit messages
3. Developer opens Pull Request targeting `main`
4. CI pipeline runs automatically (lint, typecheck, test, build)
5. At least one team member reviews the PR
6. After approval and passing CI, PR is merged to `main`

Configuration Items

Item	Location	Owner	Change Frequency
Backend source code	`backend/app/`	Harsh Kumar	High
Frontend source code	`frontend/src/`	Parag Jindal	High
ML training scripts	`ml/scripts/`	Harsh Kumar	Medium
ML model artifacts	`ml/model_artifacts_global/`	Harsh Kumar	Low (weekly retrain)
Docker configuration	`docker-compose.yml`, `docker/`	Harsh Kumar	Low
CI/CD pipelines	`.github/workflows/`	Harsh Kumar	Low
Database migrations	`backend/alembic/versions/`	Harsh Kumar	Medium
Environment config	`.env` (not committed)	All	Low
Documentation	`Documentation/`	Zane Tessmer	Medium

Environment Management

Environment	Purpose	Access
Local (Docker Compose)	Development and testing	All developers
CI (GitHub Actions)	Automated testing	Automated on push/PR
HPC (DRAC Fir Server)	GPU model training	Authorized team members

Change Control Process

1. All changes require a Pull Request
2. CI must pass before merge
3. Breaking changes require team discussion in standup
4. Database schema changes require Alembic migration
5. Model artifact changes are backed up with timestamps by Airflow

12. Change Request Document

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Change Request #1

Field	Details
CR ID	CR-001
Title	Remove authentication module from MVP
Requested By	Harsh Kumar (Product Owner)
Date Submitted	February 10, 2026
Priority	Medium
Status	Approved

Description: Remove the authentication module (JWT-based login/signup) from the Sprint 4 scope to prioritize core prediction features. Authentication will be re-implemented in Sprint 5 using JWT tokens.

Justification: The client prioritized prediction accuracy and confidence scoring over authentication for the MVP demo. The existing auth module was built on a template and needed significant rework to integrate with the market/inference modules.

Impact: Users can access all endpoints without authentication. CORS is set to allow all origins. This is acceptable for the development/demo phase but must be addressed before any production deployment.

Approval: Approved by team consensus on February 10, 2026.

Change Request #2

Field	Details
CR ID	CR-002
Title	Add prediction confidence scores to API
Requested By	Client (stakeholder feedback)
Date Submitted	February 10, 2026

Priority	High
Status	Approved & Implemented

Description: Add a confidence score (0.0 to 1.0) to each prediction response. The score should be based on market conditions (volatility, RSI, volume) and prediction characteristics (return magnitude).

Justification: Client feedback from Sprint 3 demo specifically requested confidence scores to help users assess forecast reliability.

Impact: New `confidence_score` field added to PredictionResponse schema. New `confidence.py` module created. Frontend updated to display color-coded confidence indicators.

Approval: Approved by Product Owner on February 10, 2026. Implemented by Dante Bertolutti on February 23, 2026.

13. User Support Documentation

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Table of Contents

1. User's Guide
2. Online Help
3. Release Notes
4. Training Materials

1. User's Guide

MarketSight is a web-based stock market prediction platform. Users access the system through a browser at `http://localhost:5173` (development) or the deployed URL. The application provides a dashboard with stock data visualization, ML-powered price predictions, and administrative features.

Key workflows:

- **Viewing Stocks:** Navigate to Dashboard > Stocks to browse active stocks with search and sector filtering
- **Stock Details:** Click a stock to view interactive OHLC charts with time range selection and chart type toggling
- **Getting Predictions:** Navigate to Dashboard > Predictions, select a stock or click "Predict All" to generate forecasts
- **Understanding Confidence:** Green (> 70%) = high confidence, Yellow (40-70%) = moderate, Red (< 40%) = low confidence
- **Admin Features:** Superusers can manage user accounts via Dashboard > Admin

2. Online Help

The application includes built-in help through:

- FastAPI auto-generated API documentation at `/docs` (Swagger UI) and `/redoc` (ReDoc)
- Tooltip hints on chart controls and prediction cards

- Error messages with actionable descriptions (e.g., "Stock not found", "Insufficient data")

3. Release Notes - v1.0.0 (Sprint 4 MVP)

New Features:

- Stock price prediction with XGBoost ML model (Optuna HPO-optimized)
- 53-feature engineering pipeline (RSI, MACD, Bollinger Bands, lag features, etc.)
- FMP data integration replacing yfinance with chunked downloads and gap filling
- Prediction confidence scoring (4-factor heuristic)
- Interactive OHLC charts (candlestick + line modes) with prediction overlay
- Batch prediction ("Predict All") functionality
- Full DRAC end-to-end training pipeline (data → features → model → API → Airflow → frontend)
- Automated data seeding and model retraining via Airflow
- Dashboard with summary statistics
- Admin user management
- User settings (profile, password, account)

Known Limitations:

- No user authentication (open access)
- No real-time WebSocket streaming
- No prediction history or accuracy tracking

4. Training Materials

- **Development Setup Guide:** See `development.md` in project root for Docker Compose setup, code quality tools, and testing commands
- **DRI Documentation:** See `Documentation/DRI.md` for HPC server access and model training instructions
- **API Reference:** Access `/docs` endpoint when backend is running for interactive API documentation

14. Developer's Guide (Updated)

Prepared by: Zane Tessmer | **Last Updated:** March 3, 2026

Architecture Diagram

The system follows a three-tier architecture:

```
+------------------+     +-------------------+     +------------------+
|    Frontend      |     |     Backend       |     |    Database      |
|  React 19 + TS   |<--->|  FastAPI + Python  |<--->|  PostgreSQL 16   |
|  Vite + Tailwind |     |  SQLAlchemy 2.0   |     |  market schema   |
|  TanStack Router |     |  XGBoost Model    |     |  stocks + prices |
+------------------+     +-------------------+     +------------------+
        |                       |
        |                +------+------+
        |                |   Airflow   |
        |                | Scheduler   |
        |                | DAGs: seed, |
        |                | retrain     |
        |                +-------------+
        |
  +-----+------+
  |   Nginx    |
  |   Proxy    |
  +------------+
```

Use Case Diagram

```
                   +--------------------------+
                   |       MarketSight        |
                   +--------------------------+
                   |                          |
  +------+         | [View Stock List]        |
  | User |-------->| [View Stock Chart]       |
  +------+         | [Get Prediction]         |
      |            | [Predict All Stocks]     |
      |            | [View Dashboard]         |
      |            | [Manage Settings]        |
      |            +--------------------------+
```

```
      |
  +-------+        +--------------------------+
  | Admin |------->| [Manage Users]           |
  +-------+        | [Add/Edit/Delete Users]  |
                   +--------------------------+
                            |
  +-----------+    +--------------------------+
  | Airflow   |--->| [Seed Market Data]       |
  | Scheduler |    | [Retrain ML Model]       |
  +-----------+    +--------------------------+
```

Project Code Description

Module	Path	Description
API Layer	`backend/app/api/`	FastAPI routers, dependency injection, health check
Market Module	`backend/app/modules/market/`	Stock data models, CRUD, service, API endpoints
Inference Module	`backend/app/modules/inference/`	ML prediction: model loading, features, confidence, API
Core Config	`backend/app/core/`	Pydantic settings, database engine, security utilities
Frontend Routes	`frontend/src/routes/`	TanStack Router file-based pages (dashboard, stocks, predictions)
Frontend Components	`frontend/src/components/`	Reusable UI: charts, tables, admin forms, settings
Frontend Services	`frontend/src/services/`	API client functions for market and inference endpoints
ML Scripts	`ml/scripts/`	Model training, data inspection, database connectivity
ML Features	`ml/features/`	53-feature engineering pipeline (RSI, MACD, SMA, Bollinger, lag features, etc.)
Airflow DAGs	`airflow/dags/`	Data seeding, model retraining, connectivity test

Testing Code Description

Test File	Type	Description
`tests/api/test_utils.py`	Unit	Health check endpoint verification
`tests/modules/test_market.py`	Integration	Full market API request/response cycle
`tests/modules/test_market_crud.py`	Unit	Database CRUD operations for stocks and prices
`tests/modules/test_market_service.py`	Unit	MarketService business logic layer
`tests/modules/test_inference.py`	Integration	Inference API with mocked model pipeline
`tests/modules/test_confidence.py`	Unit	All confidence scoring sub-functions and edge cases
`tests/conftest.py`	Fixture	Shared DB session and FastAPI TestClient setup

`tests/modules/conftest.py`	Fixture	Market test data: 4 stocks + 5 days OHLC seeding

Continuous Integration Report

Pipeline: GitHub Actions (`ci.yml`) **Trigger:** Push or PR to `main` branch **Last Run:** Passing (all 6 jobs green)

Job	Status	Duration
backend-lint	Passing	~30s
backend-typecheck	Passing	~45s
backend-test	Passing	~2min
frontend-lint	Passing	~20s
frontend-build	Passing	~45s
docker-build	Passing	~3min

Total pipeline time: ~7 minutes **Test results:** All tests passing (market CRUD, market API, market service, inference API, confidence scoring, health check)

Part C - Testing & Acceptance

15. Acceptance Test Results

Tested by: Zane Tessmer, Dante Bertolutti **Date:** March 1, 2026

Acceptance Test Cases

Test ID	Test Description	Expected Result	Actual Result	Status
AT-01	Navigate to /dashboard and verify summary cards load	4 summary cards displayed (stocks, sectors, model, API)	Cards displayed correctly	PASS
AT-02	Navigate to /dashboard/stocks and search for "AAPL"	AAPL stock appears in filtered list	AAPL found and displayed	PASS
AT-03	Click AAPL stock and verify candlestick chart renders	Interactive candlestick chart with OHLC data	Chart rendered correctly	PASS
AT-04	Toggle chart to line mode	Chart switches to line series	Line chart displayed	PASS
AT-05	Select 1W time range on chart	Chart zooms to last 7 days of data	Time range applied correctly	PASS
AT-06	Enable prediction overlay on chart	Dashed orange line from last price to predicted price	Overlay rendered correctly	PASS
AT-07	Navigate to /dashboard/predictions	Predictions page with stock selector loads	Page loaded correctly	PASS
AT-08	Select AAPL and get prediction	Prediction card shows symbol, prices, return %, confidence	All fields populated correctly	PASS
AT-09	Verify confidence score is between 0 and 1	Score displayed as percentage (0-100%)	Score: 72% (within range)	PASS
AT-10	Click "Predict All" button	Predictions generated for all active stocks	5 prediction cards displayed	PASS
AT-11	Verify confidence color coding	Green > 70%, Yellow 40-70%, Red < 40%	Colors matched thresholds	PASS
AT-12	Hit /api/v1/inference/predict/INVALID	404 error returned	404 "Stock not found"	PASS
AT-13	Hit /api/v1/utils/health-check/	Returns true with 200 status	Response: true, status: 200	PASS
AT-14	Run full CI pipeline on main branch	All 6 jobs pass	All jobs green	PASS

AT-15	Docker compose up and verify all services start	7 services running and healthy	All services running	PASS

Acceptance Sign-Off

All 15 acceptance test cases passed. The system meets the Sprint 4 acceptance criteria for the MVP release.

Signed:

Client Representative: _____ Date: _____

Product Owner (Harsh Kumar): _____ Date: _____

Scrum Master (Zane Tessmer): _____ Date: _____